

Zadanie 2 - Wyznaczanie powierzchni widocznych

Grafika Komputerowa

Martyna Kochalska, 331704

Jakub Próchnicki, 331730

1. Cel projektu.....	1
2. Wybrane technologie.....	2
3. Sterowanie.....	2
4. Użyte algorytmy.....	2
Wybrany układ.....	2
Eliminacja elementów zasłoniętych.....	3
5. Testy.....	6

1. Cel projektu

Celem projektu była rozbudowa dotychczasowej kamery 3D o eliminację elementów zasłoniętych. W tym celu dotychczasowy obrys krawędziowy obiektów należało wypełnić ścianami, a je obsłużyć odpowiednim algorytmem rysowania sceny. Dodatkowo, w ramach wygody testowania, zaimplementowana została możliwość poruszania się w sposób płynny, przy przytrzymywaniu przycisków, oraz o restart pozycji kamery (sceny).

Na drugim etapie prac wprowadzono, zgodnie z zaleceniami prowadzącego, następujące uproszczenia:

- Brak skomplikowanego przycinania, obiekty na scenie nie tworzą cykliów.
- Figury nie przenikają się, mogą jedynie stykać się ze sobą.

W ramach sceny wygenerowano zestaw 5 prostopadłościanów imitujących "domki" oraz jedną długą płaszczyznę imitującą "płotek".

2. Wybrane technologie

Do stworzenia aplikacji wykorzystano język Python wraz z biblioteką Pygame, służącą do rysowania punktów i linii, oraz NumPy do wykonywania obliczeń macierzowych. Kod napisano i uruchomiono w środowisku Visual Studio Code.

3. Sterowanie

Translacja:

Lewa strzałka / Prawa strzałka - ruch w lewo / prawo
Górna strzałka / Dolna strzałka - ruch do przodu / do tyłu
Shift / Spacja - ruch w dół / w górę

Obrót:

W / S - pochylenie (oś X)
A / D - odchylenie (oś Y)
Q / E - przechylenie (oś Z)

Pozostałe:

= / - - przybliżenie / oddalenie
R - reset pozycji sceny
ESC - wyjście z programu

4. Użyte algorytmy

Wybrany układ

W zaimplementowanym rozwiązaniu zastosowano model, w którym wirtualna kamera jest statyczna i zawsze znajduje się w początku układu współrzędnych (punkt 0, 0, 0). Zarówno operacje ruchu jak i obrotu realizowane są poprzez odpowiednie przekształcanie całej sceny w kierunku odwrotnym do pożądanego wektora ruchu kamery.

Eliminacja elementów zasłoniętych

Początkowo do rozwiązania problemu widoczności wybraliśmy algorytm malarski. W pierwszej fazie testów zastosowaliśmy podejście naiwne,

opierające się na sortowaniu wielokątów względem ich skrajnych punktów na osi Z. Zgodnie z przewidywaniami, metoda ta okazała się niewystarczająca dla bardziej złożonych układów, a wynikające z tego błędy w kolejności wyrysowywania zaprezentowane zostały w poniższym rozdziale „Testy” na rysunkach 2 i 3.

Następnie podjęliśmy próbę rozszerzenia metody do pełnej wersji algorytmu Newella, Newella i Sanchy, poprzez próbę implementacji zestawu odpowiednich testów geometrycznych (m.in. badania nakładania się prostokątów ograniczających). Z uwagi na trudności implementacyjne i brak jednoznacznej poprawy dla skrajnych przypadków, zdecydowaliśmy się zmienić koncepcję i podjęliśmy próbę implementacji algorytmu drzewa BSP.

Budowa Drzewa BSP

W zaimplementowanym rozwiązaniu, zgodnie z tradycyjnym podejściem drzewo budowane jest wyłącznie raz przy starcie programu.

Jako korzeń wybierana jest ściana znajdująca się w środku listy. Następnie uzyskiwana jest płaszczyzna zawierająca korzeń. Aby ją wyznaczyć, wybierane są trzy pierwsze wierzchołki wielokąta V_0 , V_1 , V_2 . Za pomocą nich wyliczane są dwa wektory leżące na płaszczyźnie:

$$\begin{aligned} \text{vec1} &= V_1 - V_0 \\ \text{vec2} &= V_2 - V_0 \end{aligned}$$

Następnie, korzystając z iloczynu wektorowego, wyliczany jest wektor normalny płaszczyzny n , który jest prostopadły do obu tych wektorów:

$$n = \text{vec1} \times \text{vec2}$$

Na koniec wektor ten poddawany jest normalizacji, poprzez podzielenie go przez jego własną długość.

$$\text{normal} = n / |n|$$

Po uzyskaniu wektora normalnego płaszczyzny, możliwe jest określenie, czy płaszczyzna przecina kolejne wielokąty. Aby tego dokonać, dla każdej ściany testowane są jej wierzchołki. Dla każdego z nich wyliczana jest odległość ze znakiem, czyli iloczyn skalarny wektora normalnego oraz wektora skierowanego od dowolnego punktu na płaszczyźnie (np. v_0) do testowanego wierzchołka:

$$\text{dist} = \text{normal} \cdot (\text{vec} - \text{v0})$$

Jeśli odległość > 0, wierzchołek leży z przodu płaszczyzny.

Jeśli odległość < 0, wierzchołek leży z tyłu płaszczyzny.

Jeśli d = 0, wierzchołek leży na płaszczyźnie.

Po tym dla każdego wielokąta liczona jest liczba wierzchołków leżących z przodu i z tyłu. Jeśli znajdują się wszystkie wyłącznie z przodu lub z tyłu, ściany dodawane są odpowiednio do list `front_faces` i `back_faces`. W pozostałych sytuacjach ściana jest przecinana przez płaszczyznę.

W tym miejscu zaimplementowany został algorytm tnący. Aby wygenerować nowe wierzchołki na krawędziach przecinających płaszczyznę, wykorzystano analityczną metodę wyznaczania punktu przecięcia odcinka z płaszczyzną. Dla każdej krawędzi łączącej wierzchołki V1 i V2 leżące po przeciwnych stronach płaszczyzny, wyliczany jest parametr t:

$$t = \text{dist1} / (\text{dist1} - \text{dist2})$$

Punkt przecięcia wyznaczany jest za pomocą równania:

$$\text{Pinter} = \text{V1} + t * (\text{V2} - \text{V1})$$

Wyliczony punkt dodawany jest do list wierzchołków obu nowo powstających fragmentów ściany. W wyniku tej operacji oryginalny wielokąt zostaje podzielony na dwa mniejsze, które są następnie dodawane do odpowiednich list `front_faces` oraz `back_faces`.

Następnie rekurencyjnie wywoływana jest dwukrotnie funkcja budowy drzewa, gdzie jako ściany podawane są listy `front_faces` oraz `back_faces`. Z tych list ponownie korzeniem zostaje element znajdujący się w ich środku, a wszystkie wyżej opisywane kroki są powtarzane na mniejszych zbiorach. Gdy wszystkie wielokąty zostaną wykorzystane do podziału, budowę drzewa można uznać za zakończoną.

Przeglądanie Drzewa BSP

Podczas działania programu, dla każdej operacji translacji i obrotu scena jest rysowana ponownie. W tym miejscu tworzona jest lista wielokątów, która będzie odpowiedzialna za ostateczne wyrysowanie klatki. Aby zapewnić

poprawne nakładanie się obiektów na płaskim ekranie monitora, obiekty na liście muszą zostać uporządkowane od najdalszego do najbliższego.

W tym celu zastosowano metodę przechodzenia przez zbudowane wcześniej drzewo drzewo BSP w kolejności INORDER±.

W zaimplementowanym rozwiązaniu wirtualna kamera znajduje się zawsze w początku układu współrzędnych. Podczas odwiedzania każdego węzła, dynamicznie wyznaczane jest równanie jego płaszczyzny na podstawie aktualnie przetransformowanych wierzchołków.

Następnie obliczana jest odległość kamery od tej płaszczyzny za pomocą iloczynu skalarnego wektora normalnego `normal` oraz wektora skierowanego od punktu na płaszczyźnie `p0` do kamery. Ponieważ współrzędne kamery to $(0,0,0)$, wektor ten można uprościć do po prostu $-p0$:

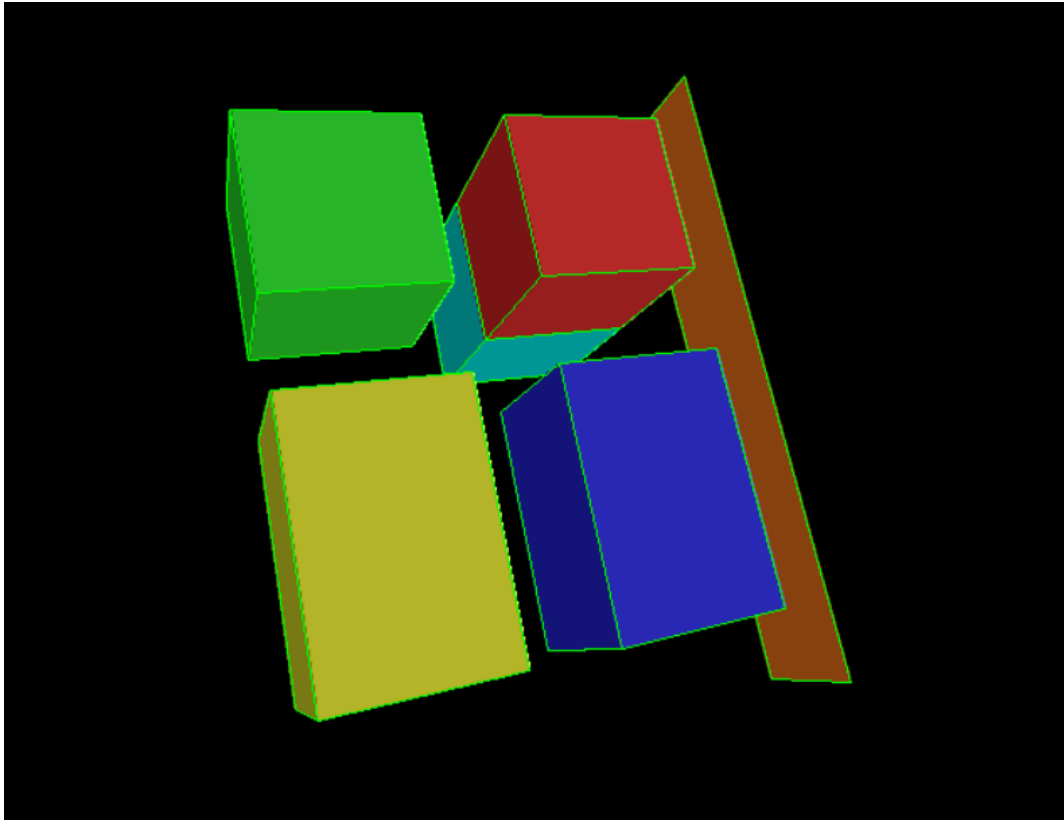
$$\text{dist} = \text{normal} \cdot (-p0)$$

Jeśli odległość > 0 , oznacza to, że obserwator znajduje się z przodu płaszczyzny. W związku z tym algorytm najpierw schodzi do poddrzewa tylnego, następnie dodaje wielokąt z bieżącego węzła do listy renderowania, a na końcu przechodzi do poddrzewa przedniego.

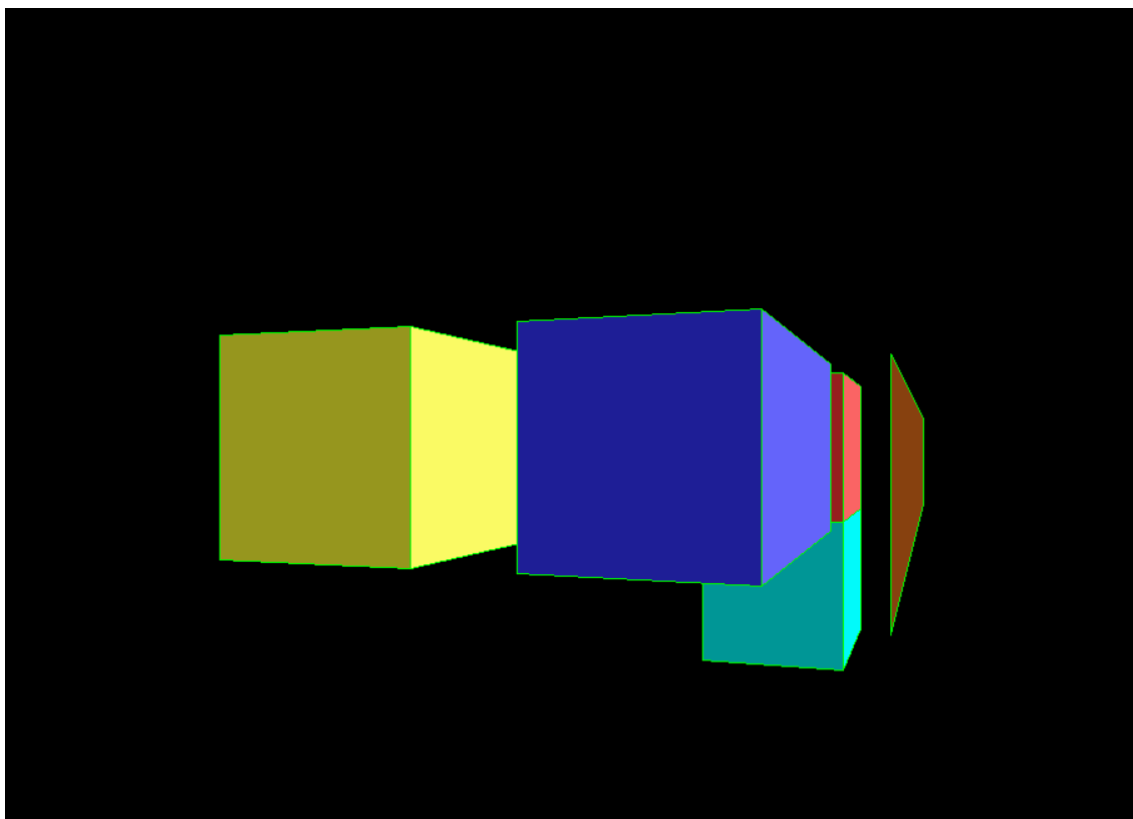
Jeśli odległość ≤ 0 , oznacza to, że obserwator znajduje się z tyłu płaszczyzny. W związku z tym algorytm w pierwszej kolejności odwiedza poddrzewo przednie, następnie dodaje bieżący wielokąt, a na końcu rekurencyjnie wchodzi do poddrzewa tylnego.

Wynikiem działania funkcji przechodzącej przez drzewo jest ostatecznie uporządkowana, jednowymiarowa lista. W końcowej fazie, wewnątrz pętli głównej rysującej scenę, wierzchołki każdego wielokąta z tej listy poddawane są weryfikacji przed błędem dzielenia przez zero, rzutowaniu perspektywicznemu oraz wyrysowaniu przy użyciu funkcji z biblioteki Pygame.

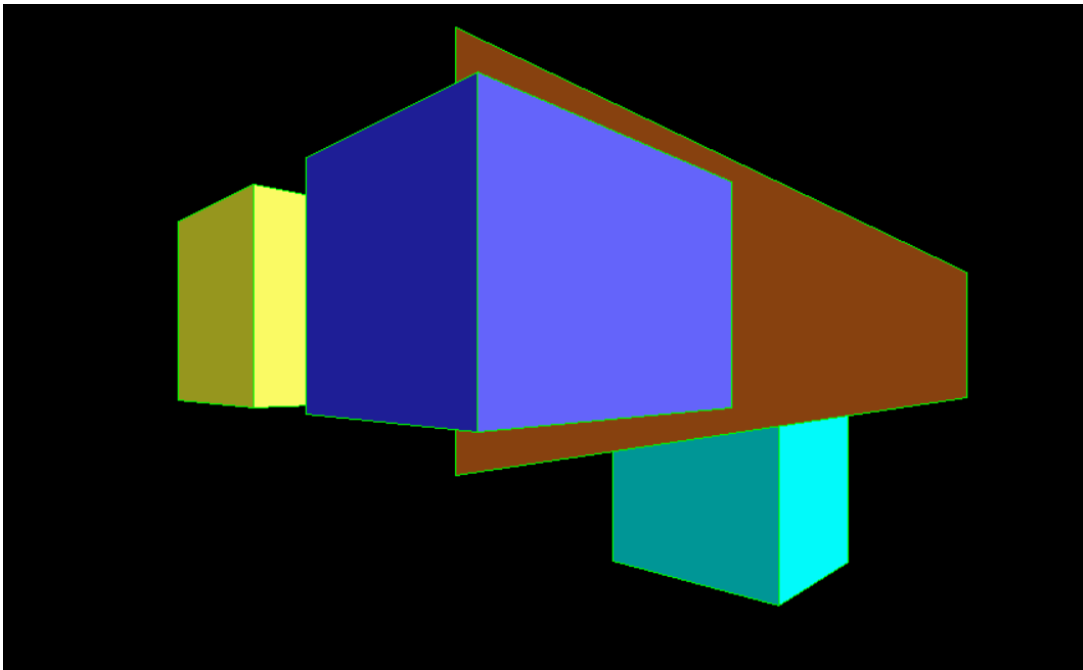
5. Testy



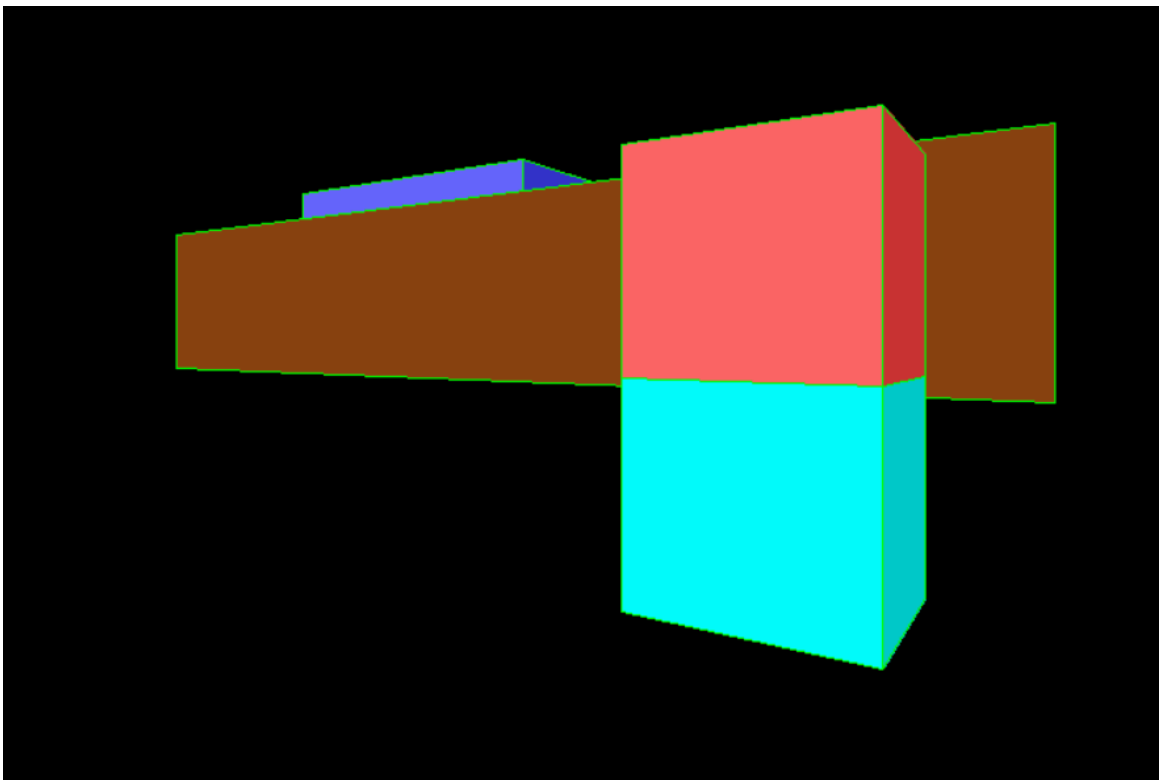
Rys. 1. Nowa scena - 5 domków i 1 płatek



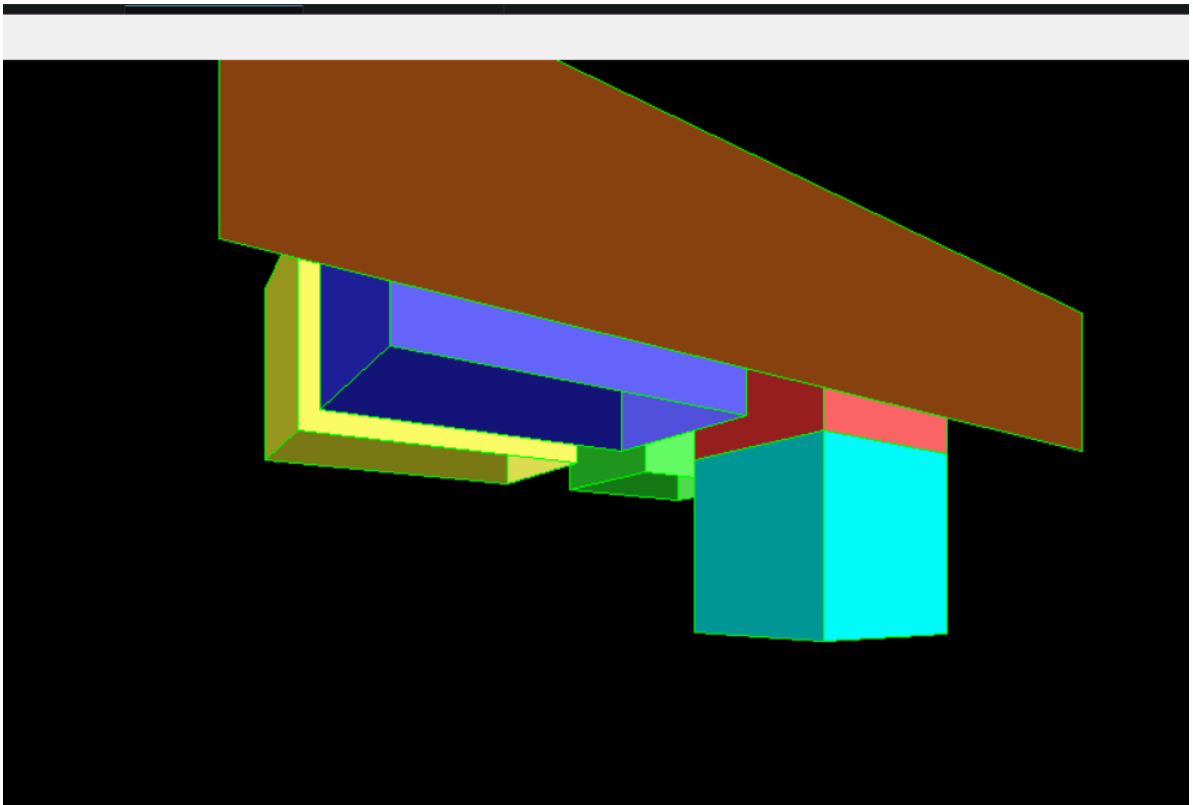
Rys. 2. Brązowa ściana znajduje się po prawej od brył niebieskich i czerwonej.



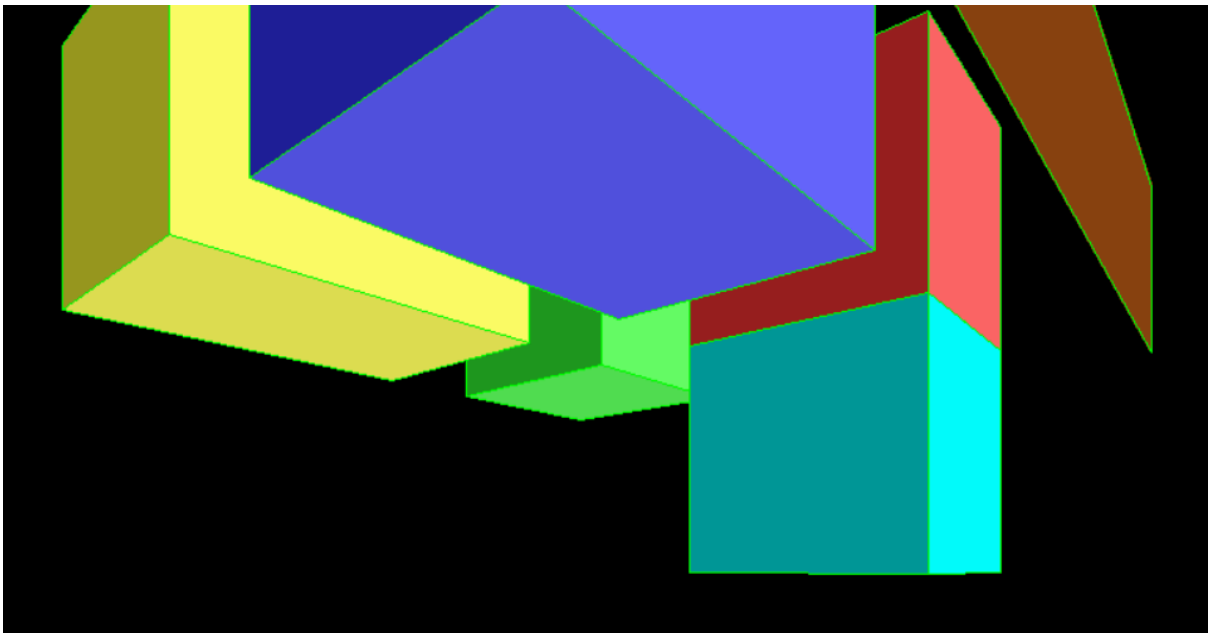
Rys. 3. Problem algorytmu malarza: zarówno w wersji uproszczonej - czystego sortowania po środkach ciężkości, jak i niedoszłej pełnej implementacji. Bryły “prześwitywały” przez brązową płaszczyznę



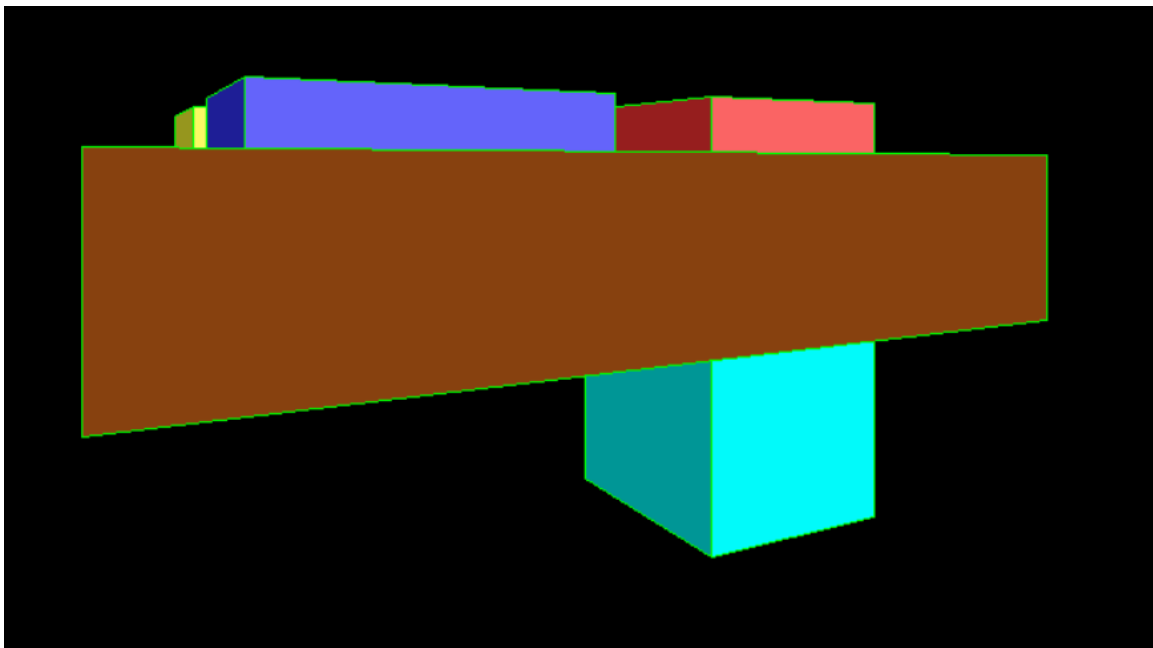
Rys. 4. Drugi przykład “prześwitywania”.



Rys. 5. Kolejny problem pierwszej implementacji - dla niektórych kątów kamery względem sceny bryły otwierały się, a niektóre ściany nie były generowane.



Rys. 6. Scena z zaimplementowanym już algorytmem drzewa BSP. Nowy, poprawnie zaimplementowany algorytm, nie ma problemu otwierania się brył.



Rys. 7. Dodatkowo naprawiony został problem prześwitywania brył przez płaszczyznę.